KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah : Pemrograman Mobile
Program Studi : D4 – Teknik Informatika
Semester : 5

Kelas : 2A TI
NIM : 244107020112
Nama : Muhammad Shabran
Pertemuan : 5

LAPORAN PRAKTIKUM

Praktikum 1: SharedPreferences

Langkah	Keterangan
1.	Persiapan Project

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> flutter create week5_offline_notes
Creating project week5_offline_notes...
Resolving dependencies in `week5_offline_notes`... (2.4s)
Downloading packages...
Got dependencies in `week5_offline_notes`.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

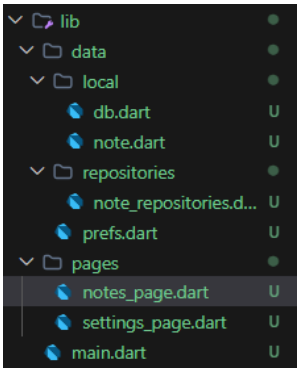
  $ cd week5_offline_notes
  $ flutter run

Your application code is in week5_offline_notes\lib\main.dart.

PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> cd week5_offline_notes
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week5_offline_notes> flutter pub add flutter_riverpo
d shared_preferences sqflite path
Resolving dependencies...
Downloading packages... (37.1s)
+ crypto 3.0.7
  cupertino_icons 1.0.9 (2.0.0 available)
+ ffi 2.2.0
+ file 7.0.1
+ fixnum 1.1.1
+ flutter_riverpod 3.4.3
+ flutter_web_plugins 0.0.0 from sdk flutter
+ listen 1.0.1
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  path 1.9.1 (from transitive dependency to direct dependency)
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ riverpod 3.4.3
+ shared_preferences 2.5.6
+ shared_preferences_android 2.4.28
+ shared_preferences_foundation 2.5.7
+ shared_preferences_linux 2.4.1
+ shared_preferences_platform_interface 2.4.2
```

```
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ riverpod 3.4.3
+ shared_preferences 2.5.6
+ shared_preferences_android 2.4.28
+ shared_preferences_foundation 2.5.7
+ shared_preferences_linux 2.4.1
+ shared_preferences_platform_interface 2.4.2
+ shared_preferences_web 2.4.3
+ shared_preferences_windows 2.4.1
+ sqflite 2.4.4+1
+ sqflite_android 2.4.4
+ sqflite_common 2.5.13+1
+ sqflite_darwin 2.4.4
+ sqflite_platform_interface 2.4.2
+ state_notifier 1.0.0
+ synchronized 3.4.2
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
+ uuid 4.6.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
+ xdg_directories 1.1.0
Changed 32 dependencies!
5 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
Building with plugins requires symlink support.

Please enable Developer Mode in your system settings. Run
  start ms-settings:developers
to open settings.
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week5_offline_notes> flutter pub add flutter_riverpo
d shared_preferences sqflite path
"flutter_riverpod" is already in "dependencies". Will try to update the constraint.
"shared_preferences" is already in "dependencies". Will try to update the constraint.
"sqflite" is already in "dependencies". Will try to update the constraint.
"path" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  test_api 0.7.12 (0.7.14 available)
```



2.	Repository preferensi

```dart
import 'package:shared_preferences/shared_preferences.dart';

class PrefsRepository {
  static const _darkModeKey = 'dark_mode';
  static const _lastOpenedKey = 'last_opened_at';

  Future<bool> getDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_darkModeKey) ?? false;
  }

  Future<void> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_darkModeKey, value);
  }

  Future<void> markOpenedNow() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_lastOpenedKey, DateTime.now().toIso8601String());
  }

  Future<String?> getLastOpened() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_lastOpenedKey);
  }
}
```

3. Provider dan halaman pengaturan

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../data/prefs.dart';

final prefsRepositoryProvider = Provider((ref) => PrefsRepository());
final darkModeProvider = AsyncNotifierProvider<DarkModeNotifier, bool>(DarkModeNotifier.new);

class DarkModeNotifier extends AsyncNotifier<bool> {
  @override
  Future<bool> build() => ref.watch(prefsRepositoryProvider).getDarkMode();

  Future<void> toggle() async {
    final next = !(state.value ?? false);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await ref.read(prefsRepositoryProvider).setDarkMode(next);
      return next;
    });
  }
}
```

Praktikum 2: SQLite dan repository catatan

Langkah	Keterangan
1.	Model catatan

```dart
class Note {
  const Note({
    this.id,
    required this.title,
    this.body = '',
    required this.updatedAt,
    this.dirty = false,
  });

  final int? id;
  final String title;
  final String body;
  final DateTime updatedAt;
  final bool dirty;

  Map<String, Object?> toMap() => {
        'id': id,
        'title': title,
        'body': body,
        'updated_at': updatedAt.toIso8601String(),
        'dirty': dirty ? 1 : 0,
      };

  factory Note.fromMap(Map<String, Object?> map) {
    return Note(
      id: (map['id'] as num?)?.toInt(),
      title: map['title'] as String? ?? '',
      body: map['body'] as String? ?? '',
      updatedAt: DateTime.tryParse(map['updated_at'] as String? ?? '') ??
          DateTime.fromMillisecondsSinceEpoch(0),
      dirty: ((map['dirty'] as num?)?.toInt() ?? 0) == 1,
    );
  }
}
```

2. Pembuka database

```dart
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

Future<Database> openNotesDb() async {
  final dir = await getDatabasesPath();
  return openDatabase(
    p.join(dir, 'offline_notes.db'),
    version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE notes(
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          title TEXT NOT NULL,
          body TEXT NOT NULL DEFAULT '',
          updated_at TEXT NOT NULL,
          dirty INTEGER NOT NULL DEFAULT 0
        )
      ''');
      await db.execute('''
        CREATE TABLE cached_posts(
          id INTEGER PRIMARY KEY,
          payload TEXT NOT NULL,
          cached_at TEXT NOT NULL
        )
      ''');
    },
  );
}
```

3. Repository sebagai satu-satunya pintu data

```dart
import 'package:sqflite/sqflite.dart';
import '../local/db.dart';
import '../local/note.dart';

class NoteRepository {
  NoteRepository({Future<Database> Function()? openDb})
      : _openDb = openDb ?? openNotesDb;

  final Future<Database> Function() _openDb;

  Future<List<Note>> fetchNotes() async {
    final db = await _openDb();
    final rows = await db.query('notes', orderBy: 'updated_at DESC');
    return rows.map(Note.fromMap).toList();
  }

  Future<Note> addNote({required String title, String body = ''}) async {
    final db = await _openDb();
    final note = Note(
      title: title,
      body: body,
      updatedAt: DateTime.now(),
      dirty: true,
    );
    final id = await db.insert('notes', note.toMap());
    return Note(
      id: id,
      title: note.title,
      body: note.body,
      updatedAt: note.updatedAt,
      dirty: true,
    );
  }

  Future<void> deleteNote(int id) async {
    final db = await _openDb();
    await db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }

  Future<int> countDirty() async {
    final db = await _openDb();
    final rows = await db.rawQuery(
        "SELECT COUNT(*) AS c FROM notes WHERE dirty = 1");
    return ((rows.first['c'] as num?)?.toInt() ?? 0);
  }

  Future<void> markAllSynced() async {
    final db = await _openDb();
    await db.update('notes', {'dirty': 0}, where: 'dirty = 1');
  }
}
```

Praktikum 3: Cache-first dan antrean sync

Langkah	Keterangan
1.	Cache-first read untuk data API

```dart
class Post {
  const Post({required this.id, required this.title, required this.body});
  final int id;
  final String title;
  final String body;

  factory Post.fromJson(Map<String, dynamic> json) {
    return Post(
      id: json['id'] as int,
      title: json['title'] as String,
      body: json['body'] as String,
    );
  }
}
```

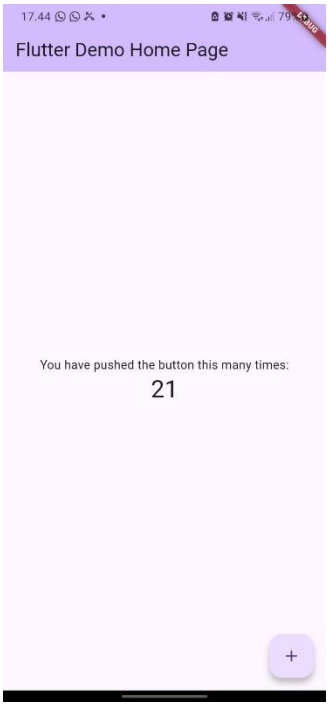
2. Sinkronisasi catatan kotor (dirty)

```dart
Future<int> syncNotes() async {
  final dirtyCount = await countDirty();
  if (dirtyCount == 0) return 0;

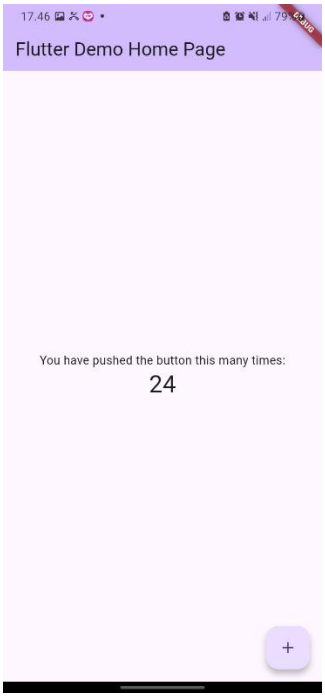
  await Future.delayed(const Duration(seconds: 1));
  await markAllSynced();
  return dirtyCount;
}
```

3. Hasil:

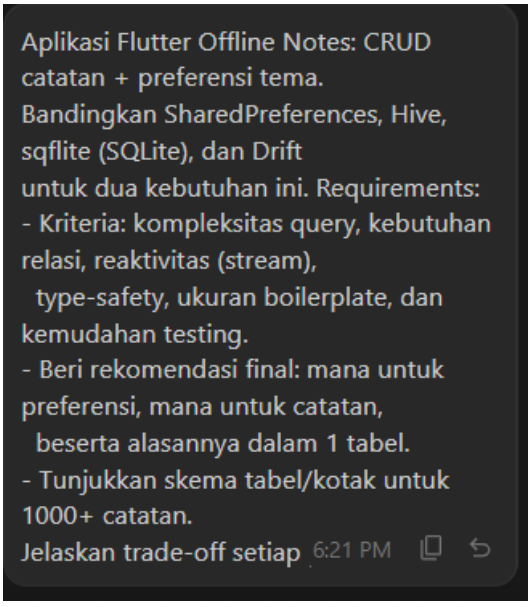
Dengan Wifi:



Tanpa WiFi:

Ai Chalengge

Langkah	Keterangan
1.	

1. Matriks Perbandingan Berdasarkan Kriteria

Kriteria	SharedPreferences	Hive (NoSQL Key-Value)	sqflite (Raw SQLite)	Drift (ORM/Type-safe SQLite)
Kompleksitas Query	Sangat rendah (hanya `get(key)` / `set(key)`)	Rendah-Sedang (filter via in-memory list atau index box)	**Sangat Tinggi** (SQL lengkap: JOIN, GROUP BY, WHERE, ORDER BY, LIMIT-OFFSET)	**Sangat Tinggi** (Fluent Dart API + SQL query dengan syntax check)
Kebutuhan Relasi	Tidak mendukung	Tidak mendukung secara native (manual via foreign key key/ID di memori)	**Didukung Penuh** (Foreign Keys, CASCADE, Joins)	**Didukung Penuh** (Relasi deklaratif, auto JOIN generator)
Reaktivitas (Stream)	Manual (perlu ValueNotifier / StreamController / Riverpod)	Ada `box.watch(key: ...)` (reaktivitas bawaan)	Manual (harus trigger provider/notifier manual saat insert/update)	**Native & Otomatis** (query langsung menghasilkan `Stream<List<T>>` auto-update saat tabel berubah)
Type-Safety	Lemah (berbasis dynamic/primitive types, rawan runtime error)	Sedang (TypeAdapter kuat untuk model, tapi query filter berbasis Dart biasa)	Lemah (menggunakan `Map<String, Object?>`, rawan salah ketik nama kolom)	**Sangat Kuat** (Compile-time verified, auto-generated Dart data class)
Ukuran Boilerplate	Sangat minimal	Sedang (`@HiveType`, `@HiveField`, `build_runner`)	Sedang (`CREATE TABLE`, manual `toMap()` dan `fromMap()`)	Tinggi diawal (perlu deklarasi tabel, `build_runner`, file `.g.dart`)
Kemudahan Testing	Sangat mudah (`setMockInitialValues`)	Mudah (bisa mock box atau write ke temporary path)	Sedang (butuh `sqflite_common_ffi` untuk unit test desktop/headless)	**Sangat Mudah** (mendukung `NativeDatabase.memory()` bawaan tanpa emulator/device)

2. Trade-off Setiap Pilihan

A. SharedPreferences

- **Kelebihan:** Sangat ringan, tidak butuh setup rumit, zero boilerplate untuk menyimpan primitif (`bool`, `int`, `String`).
- **Kekurangan:** Seluruh data dibaca ke memori di awal startup (XML di Android / Plist di iOS). Operasi tulis asynchronous tapi mem-block disk I/O jika datanya besar.
- **Trade-off:** Ideal untuk 5–20 nilai konfigurasi, namun menjadi bencana performa jika dipaksakan menyimpan list objek JSON catatan.

B. Hive

- **Kelebihan:** Performa baca/tulis tercepat karena berbasis file binary dan load index ke RAM. Tanpa dependency native (pure Dart), sangat ramah cross-platform.
- **Kekurangan:** Semua data yang di-query harus ditampung di RAM (kecuali lazy box, namun lazy box kehilangan kecepatan). Tidak ada engine SQL, sehingga sorting dan filtering 1.000+ data harus dilakukan di memori CPU (Boros RAM).
- **Trade-off:** Cepat untuk data semi-terstruktur, namun berisiko memory leak/lag jika catatan bertambah banyak beserta lampirannya.

C. sqflite

- **Kelebihan:** Standar industri, langsung mengakses mesin SQLite C-native di platform, efisien dalam konsumsi memori berkat pagination (`LIMIT` dan `OFFSET`), indexing B-Tree sangat kencang.
- **Kekurangan:** Manual mapping `Map<String, dynamic>` rawan typo nama kolom (misal: `'updated_at'` vs `'updatedAt'`), migrasi skema manual rawan human error, tidak ada streaming reaktif bawaan.
- **Trade-off:** Kontrol penuh terhadap query SQL mentah, tetapi produktivitas developer lebih rendah karena raw query boilerplate.

D. Drift

- **Kelebihan:** Memberikan kekuatan penuh SQLite dengan **Compile-time safety**. Menghasilkan `Stream` reaktif otomatis saat ada perubahan baris data, migrasi skema teruji rapi, dan mudah di-test dengan SQLite in-memory.
- **Kekurangan:** Menambahkan dependensi `build_runner` yang membuat waktu build lebih lama, kurva belajar DSL Drift lebih tinggi dibanding raw SQL biasa.
- **Trade-off:** Biaya setup dan build time lebih tinggi, tetapi menjamin bug runtime mendekati 0 untuk aplikasi berskala produksi.

3. Skema Tabel / Kotak untuk 1000+ Catatan

Untuk menangani **1.000+ catatan** secara efisien (responsif, offline sync queue, dan pagination instan), skema **HARUS** memiliki indeks pada kolom filter/sorting (`updated_at` dan `dirty`).

Opsi 1: Skema SQLite Relasional (sqflite / Drift) — *Sangat Direkomendasikan*

```sql
-- Tabel Catatan
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    body TEXT NOT NULL DEFAULT '',
    updated_at INTEGER NOT NULL,          -- Simpan epoch milliseconds untuk sorting cepat
    dirty INTEGER NOT NULL DEFAULT 0      -- 1 = belum sync ke server, 0 = synced
);

-- Indeks krusial untuk pagination cepat: SELECT * FROM notes ORDER BY updated_at DESC LIMIT 20 OFFSET 0;
CREATE INDEX idx_notes_updated_at ON notes(updated_at DESC);
```

0 Files With Changes — Review Changes

Testing

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week5_offline_notes> flutter analy
ze
Analyzing week5_offline_notes...
No issues found! (ran in 18.7s)
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week5_offline_notes> flutter test
00:05 +5: All tests passed!
```

Refleksi

- Mengapa daftar catatan tidak boleh disimpan di SharedPreferences? Apa yang rusak jika aturan ini dilanggar?

  Answer:

  SharedPreferences hanya dirancang untuk data primitif kecil. Menyimpan daftar catatan besar (JSON) di sana akan memicu aplikasi lambat dan *crash*, karena sistem harus membaca dan menulis ulang seluruh string JSON untuk setiap perubahan kecil.

- Kapan cache-first cukup, dan kapan Anda membutuhkan strategi lain (misalnya network-first untuk data harga real-time)?

  Answer:

  Cache-first cukup untuk data yang jarang berubah dan mengutamakan kecepatan tampil di layar (misal: daftar artikel atau *feed*). *Network-first* wajib digunakan untuk data kritis yang menuntut akurasi *real-time* (misal: saldo bank atau harga saham).

- Bagaimana dirty flag berubah menjadi antrean sync tanpa memblokir UI? Kapan antrean terpisah (tabel outbox) menjadi perlu?

  Answer:

  Dirty flag disinkronkan oleh fungsi *asynchronous* yang berjalan di latar belakang (*background thread*), sehingga UI tetap responsif. Tabel antrean terpisah (*outbox*) baru diperlukan jika urutan mutasi data sangat krusial (misal: operasi *create* harus selesai sebelum *update*) atau melibatkan relasi banyak tabel.

- Bagian mana dari rekomendasi AI yang Anda tolak, dan mengapa?

  Answer:

  Saya menolak saran AI untuk memisahkan repository Post dan Note ke dalam file yang berbeda. Saya memilih menggabungkannya agar proses pemanggilan provider dan penulisan kode lebih ringkas untuk skala praktikum ini.